

# Computer Networks – Experiment – TCP

**Dr. Feng Hong**

[gray.hong@gmail.com](mailto:gray.hong@gmail.com)

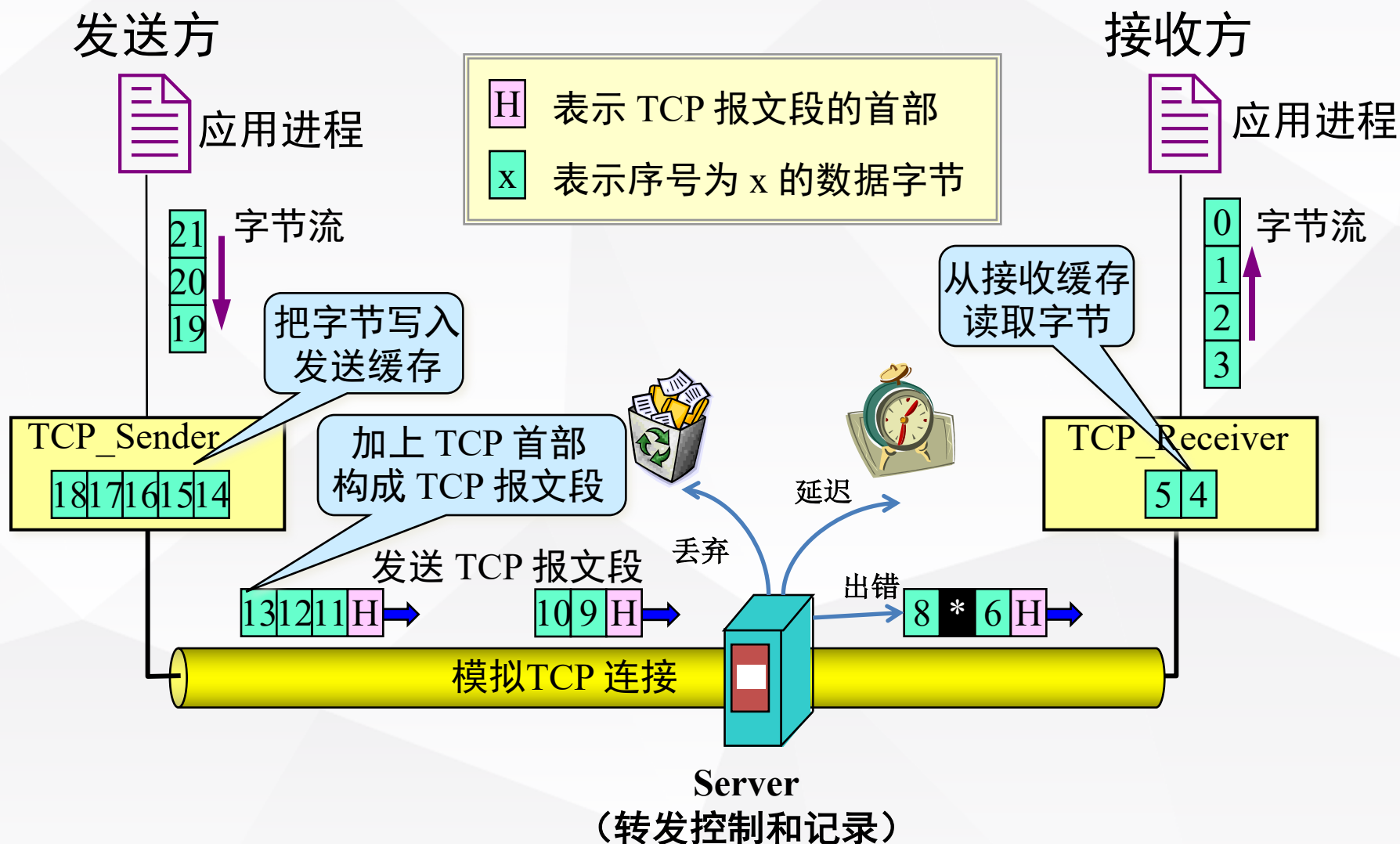
<http://osn.ouc.edu.cn/~hong>

信息学院南楼B517房间



**中国海洋大学**  
OCEAN UNIVERSITY OF CHINA

# TCP实验系统工作原理



# 实验内容

- 实现**TCP**协议端到端的**可靠传输**：数据包传送和确认，及对各种传输错误的处理；优秀：选择响应
- Java编程：实现TCP\_Sender类和TCP\_Receiver类，实现：RDT2.1（4+2分） -> RDT2.2（7+2分） -> RDT3.0（10+2分） -> Go-Back-N（13+3分）或 选择响应协议（14+3分） -> 拥塞控制（17分+3分） -> **TCP Reno版本（19分+3分）**：使用1个计时器、累积确认、用死啊控制
- 实验报告要求：从实验数据文件中自己说明实现了相关功能
- 调试：设置TCP首部中的eFlag指明支持处理的错误类型，包括：
  - 0.信道无差错
  - 1.只出错
  - 2.只丢包
  - 3.只延迟
  - 4.出错 / 丢包
  - 5.出错 / 延迟
  - 6.丢包 / 延迟
  - 7.出错 / 丢包 / 延迟
- 服务器端Log日志：记录所有数据包的传输（DATA和ACK）
- 成绩评定：实验报告 + Log文件 + recvData文件 + 代码包
- 评定准则：报告通过 代码及Log 文件证明实现了每项要求的功能

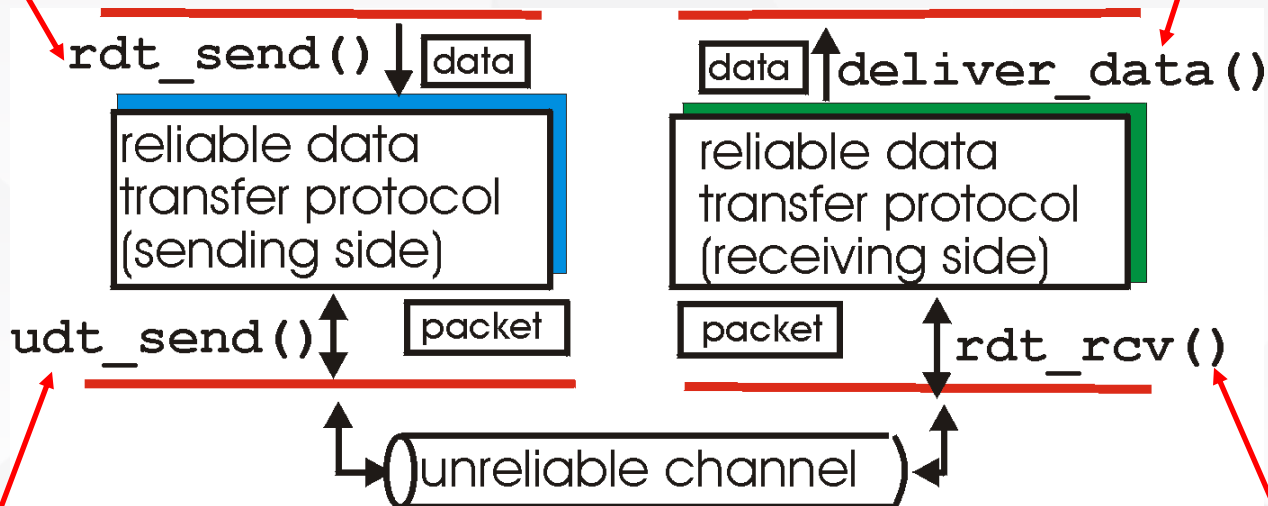
# 可靠数据传输：信道模型

**rdt\_send():** 由上层进行调用 (如应用进程)。将数据传入发送方并由其传给接收方的上层

**deliver\_data():** 由 rdt调用将数据递交给上层

发送方

接收方



**udt\_send():** 由rdt调用, 将分组通过不可靠的信道传到接收方

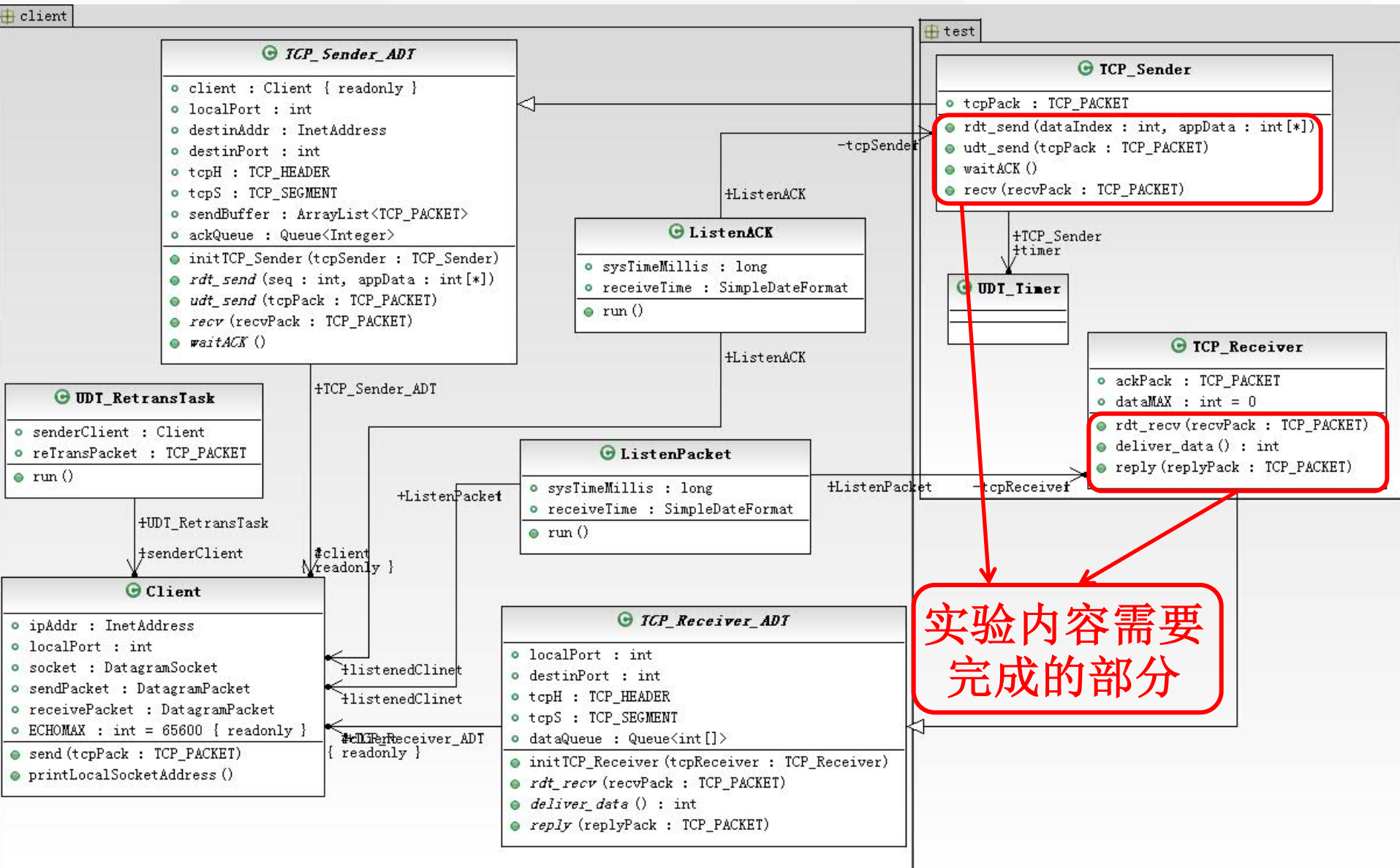
**rdt\_rcv():** 当分组到达接收方时调用



# 系统类库UML图



TCP\_TestSys.jar  
Executable Jar File  
33 KB



# 计时器的使用

- 多线程：一条线程指的是进程中一个单一顺序的控制流，一个进程中可以并发多个线程，每条线程并行执行不同的任务。
- 超时重传：计时器UDT\_Timer控制一个独立于发送方主线程的重传任务线程UDT\_RetransTask，以固定周期（如：3秒）执行重传任务。
- 计时器及重传任务的创建与调用：

```
//设置计时器和超时重传任务
```

```
UDT_Timer timer = new UDT_Timer();
```

```
UDT_RetransTask reTrans = new UDT_RetransTask(client, tcpPack);
```

- 该线程启动后会每隔3s发送一次

```
//每隔3秒执行重传，直到收到ACK
```

```
timer.schedule(reTrans, 3000, 3000);
```

- 已经成功传送包的计时器必须手动结束

```
@Override
```

```
//等待期望的ACK报文
```

```
public void waitACK() {
```

```
    //循环检查确认号对列中是否有新收到的ACK
```

```
    while(true) {
```

```
        if(!ackQueue.isEmpty() && ackQueue.poll() == tcpPack.getTcpH().getTh_seq()) {
```

```
            timer.cancel();
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

# 成绩评定

- 实验报告 + 代码 + Log文件 + recvData文件



| CLIENT        | HOST                    | TOTAL | SUC_RATIO     | NORMAL | WRONG | LOSS | DELAY |
|---------------|-------------------------|-------|---------------|--------|-------|------|-------|
| 192.168.1.105 | :9001                   | 1017  | 98.33%        | 1009   | 2     | 4    | 2     |
|               | 2012-12-19 12:22:19:453 | CST   | DATA_seq: 1   | ACKed  |       |      |       |
|               | 2012-12-19 12:22:19:531 | CST   | DATA_seq: 101 | ACKed  |       |      |       |
|               | 2012-12-19 12:22:19:562 | CST   | DATA_seq: 201 | ACKed  |       |      |       |
|               | 2012-12-19 12:22:19:609 | CST   | DATA_seq: 301 | ACKed  |       |      |       |
|               | 2012-12-19 12:22:19:625 | CST   | DATA_seq: 401 | ACKed  |       |      |       |
|               | 2012-12-19 12:22:19:656 | CST   | DATA_seq: 501 | ACKed  |       |      |       |

...

|                         |     |                      |        |
|-------------------------|-----|----------------------|--------|
| 2012-12-19 12:22:26:515 | CST | DATA_seq: 11201      | ACKed  |
| 2012-12-19 12:22:26:531 | CST | DATA_seq: 11301      | ACKed  |
| 2012-12-19 12:22:26:546 | CST | DATA_seq: 11401      | ACKed  |
| 2012-12-19 12:22:26:562 | CST | DATA_seq: 11501      | LOSS   |
| 2012-12-19 12:22:31:562 | CST | *Re: DATA_seq: 11501 | ACKed  |
| 2012-12-19 12:22:31:562 | CST | DATA_seq: 11601      | ACKed  |
| 2012-12-19 12:22:31:578 | CST | DATA_seq: 11701      | ACKed  |
| 2012-12-19 12:22:31:593 | CST | DATA_seq: 11801      | ACKed  |
| 2012-12-19 12:22:31:609 | CST | DATA_seq: 11901      | ACKed  |
| 2012-12-19 12:22:31:625 | CST | DATA_seq: 12001      | NO_ACK |
| 2012-12-19 12:22:36:625 | CST | *Re: DATA_seq: 12001 | ACKed  |
| 2012-12-19 12:22:36:640 | CST | DATA_seq: 12101      | ACKed  |
| 2012-12-19 12:22:36:640 | CST | DATA_seq: 12201      | ACKed  |
| 2012-12-19 12:22:36:656 | CST | DATA_seq: 12301      | ACKed  |

...

...

944987  
945031  
945037  
945059  
945089  
945103  
945143  
945151  
945179  
945209  
945211  
945227  
945233  
945289  
945293  
945331  
945341  
945349  
945359

...

# 挑战：拥塞控制

- 代码要求：

- 必须：慢开始、加法增大、拥塞避免、乘法减小，即TCP Tahoe版本
- 可选：快重传、快恢复，1个计时器，即TCP Reno版本

- 报告要求

- 必须在报告中根据代码和运行的Trace文件说明“慢开始、加法增大、拥塞避免、乘法减小”如何实现及其在Trace文件中的反映

# 实验环境配置

- 安装jdk-6-6u45-windows-i586 .exe
  - 课程资源库里有
  - 注意必须使用32位jdk6，因为有部分类和后续的J2SE版本不兼容
- 配置windows环境变量
  - 参考以下的网页照做即可
  - <https://jingyan.baidu.com/article/925f8cb836b26ac0dde0569e.html>

# IDE配置

- 推荐使用ECLIPSE
  - 课程资源库里
    - 解压缩eclipse-java-luna-SR2-win32.zip
    - 启动Eclipse
  - 在Eclipse中的Window下的Preference里的Java->Installed JREs中添加你安装的JRE6，并选为默认的JRE

# Eclipse配置

- 解压出TCP\_Test\_2020.rar
- 将工程根目录TCP\_Test导入Eclipse（勾选Copy projects into workspace）
- 将TCP\_TestSys.jar类库加入到Project Properties的Java Build Path的Libraries的Add JARs中
  - 加入后整个代码里不在显示错误符号
- 运行TestRun.java检验是否配置成功
- 由于运行后会绑定TCP端口，因此每次于运行结束后，需要手动在Console界面结束程序，释放TCP端口。

# Go-Back-N用到的Retrans类

```
package com.ouc.tcp.test;

import java.util.TimerTask;

/*****
 *
 * @author gray
 * improve from TransWindow : use packets directly
 */
public class TaskPacketsRetrans extends TimerTask {

    private Client senderClient;
    private TCP_PACKET[] packets;

    /*构造函数*/
    public TaskPacketsRetrans(Client client, TCP_PACKET[] packets4Retrans) {
        super();
        senderClient = client;
        packets = packets4Retrans;
    }

    @Override
    /*重传TCP数据报*/
    public void run() {

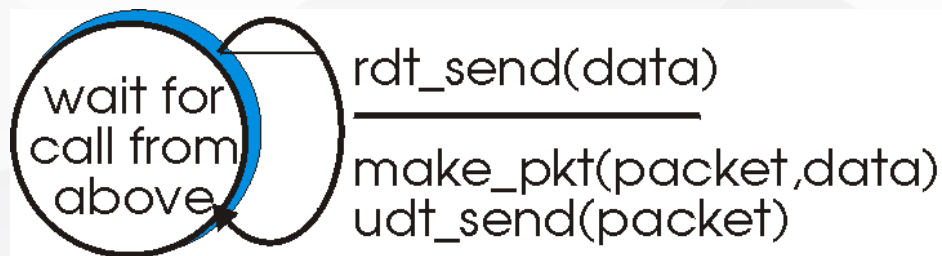
        for (int i=0;i<packets.length;i++)
            senderClient.send(packets[i]);
    }
}
```



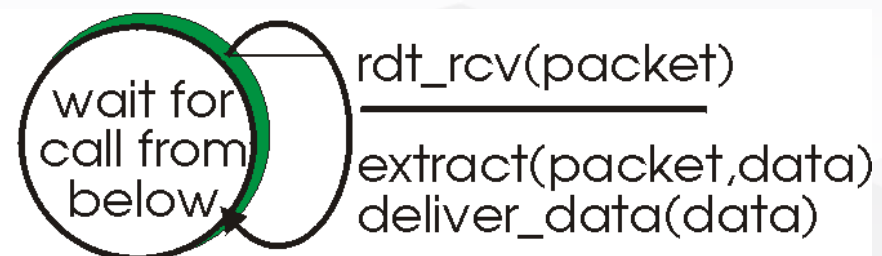
# RDT1.0: 在可靠信道上进行可靠的数据传输

- **假设:** 所依赖的信道非常可靠
  - 不可能有位错
  - 不会丢失数据
- **方法:** 分别为发送方和接收方建立 FSMs
  - 发送方将数据送入所依赖的信道
  - 接收方从所依赖的信道读出数据

例子: Sender 1.0  
Receiver 1.0



(a) rdt1.0: sending side



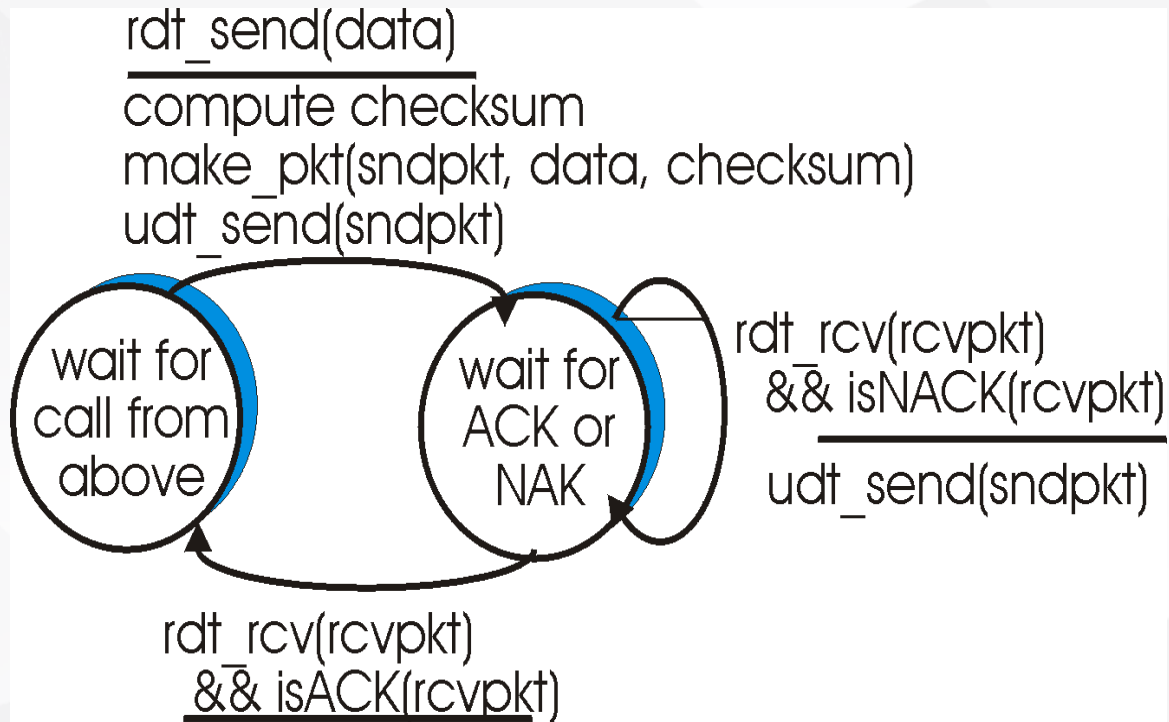
(b) rdt1.0: receiving side

# RDT2.0: 信道上可能出现位错

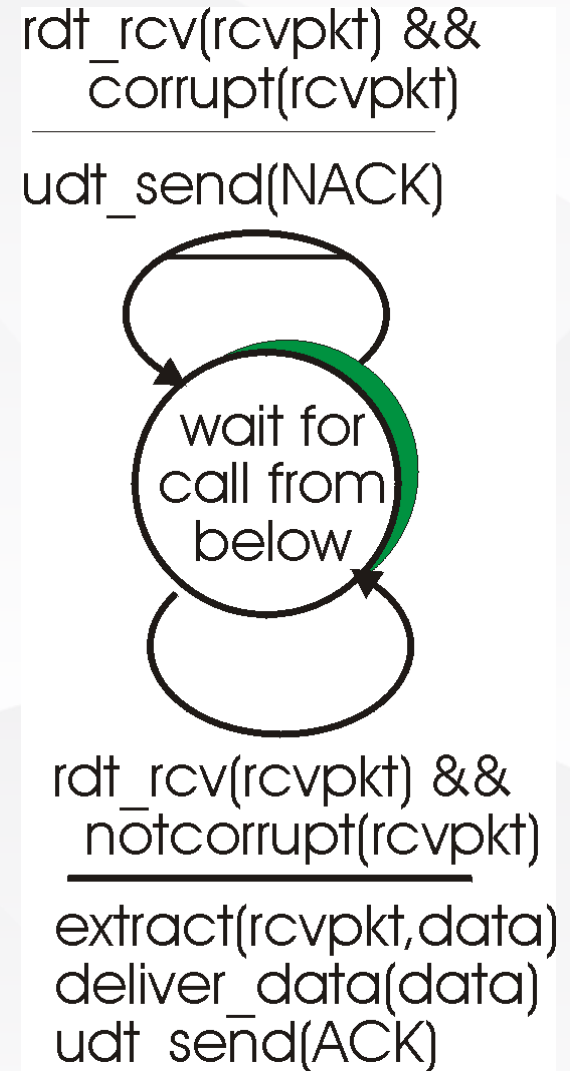
- 假设：
  - 底层信道传输过程中，个别数据包的某些字节可能发生位错
- 方法：
  - 使用校验和算法检查数据包的正确性（发送前计算/接收后校验）
  - 完成Checksum类中的computeChkSum()方法  
(注意：只需校验TCP首部中的seq、ack和sum，及TCP数据字段；校验和计算涉及数据类型的转换，而Java中没有unsigned类型，需要特殊处理)

# RDT2.0: 有限状态机定义

## 发送方FSM



## 接收方FSM



# RDT2.0 有一个致命的缺点!

## 若ACK/NAK 报文出错?

- 发送方将不会知道接收端发生了什么!
- 假如进行重传:可能发生数据重复

## 怎么办?

- 发送 ACK/NAK 来回应接收方的 ACK/NAK?
  - 那么如果发送方的 ACK/NAK 丢失?
- 重传, 但可能导致重传了正确的分组!

## 管理重复的问题:

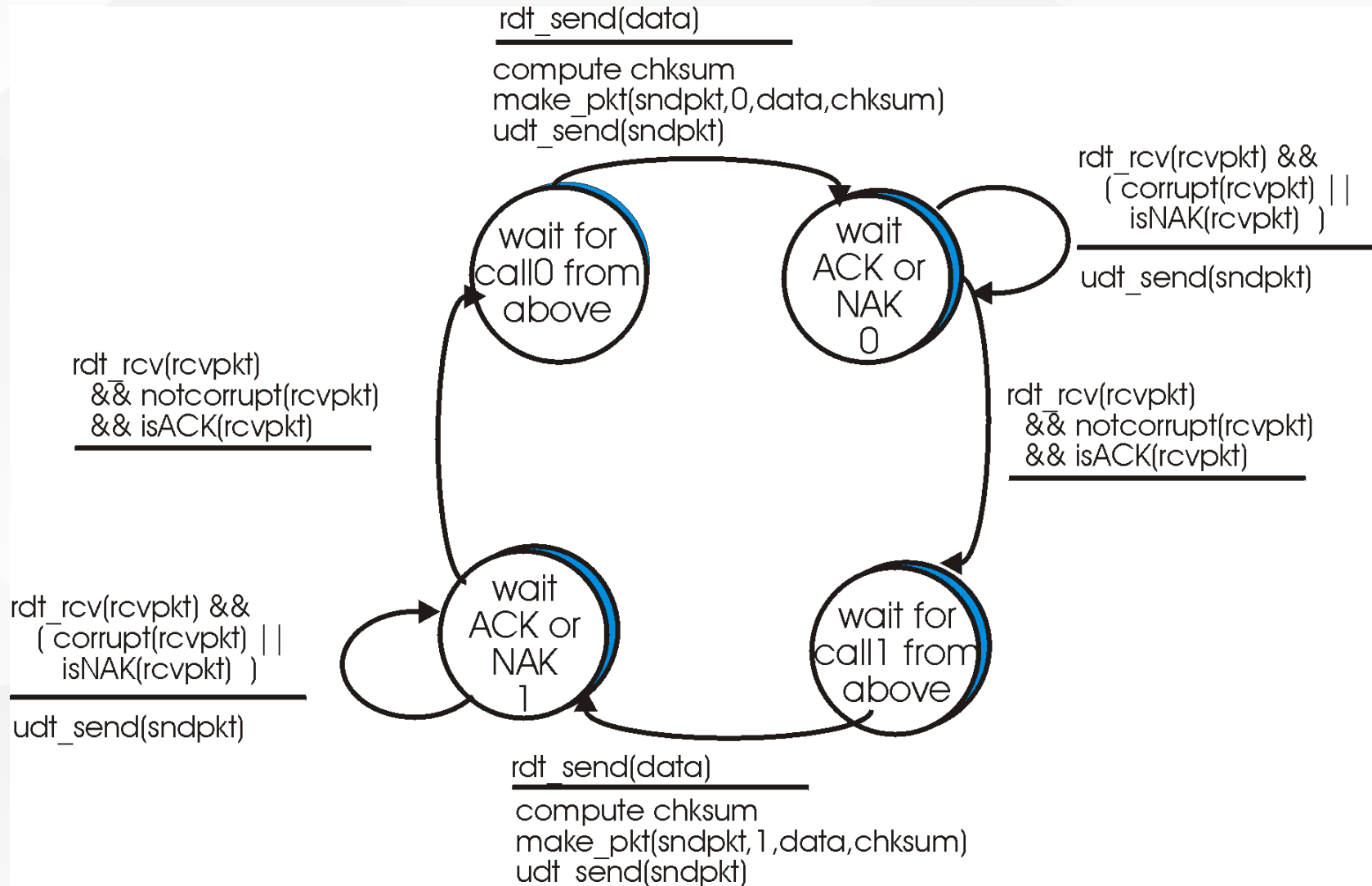
- 发送方给每个分组加上 sequence number (序号)
- 如果ACK/NAK丢失出错, 发送方则重传当前分组
- 接收方丢弃重复的分组 (不向上递交)

### 停等策略

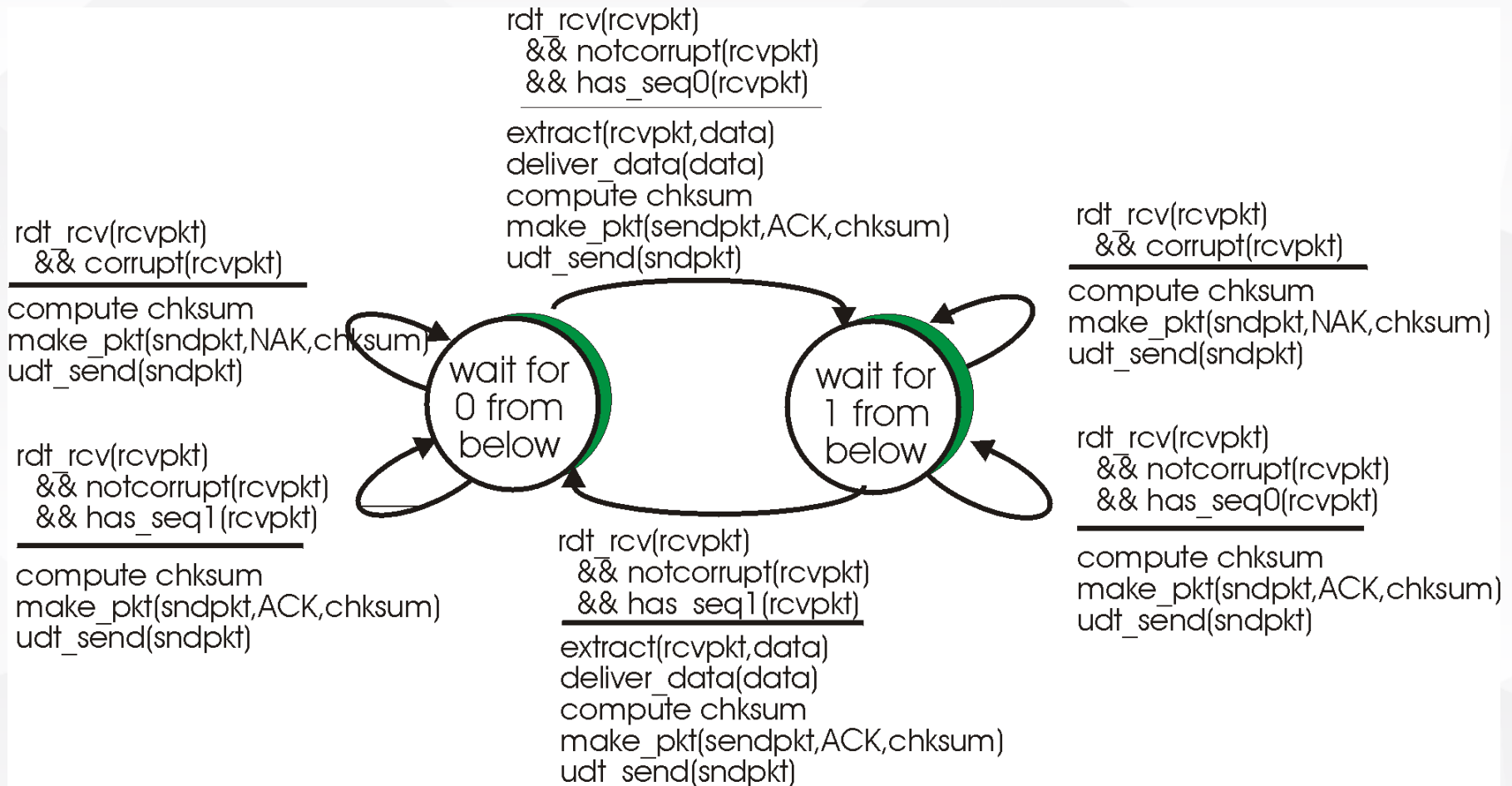
发送方法送一个分组, 然后等待接收方的响应

例子代码: CDT 2.1

# RDT2.1: 管理出错的ACK/NAK, 发送方



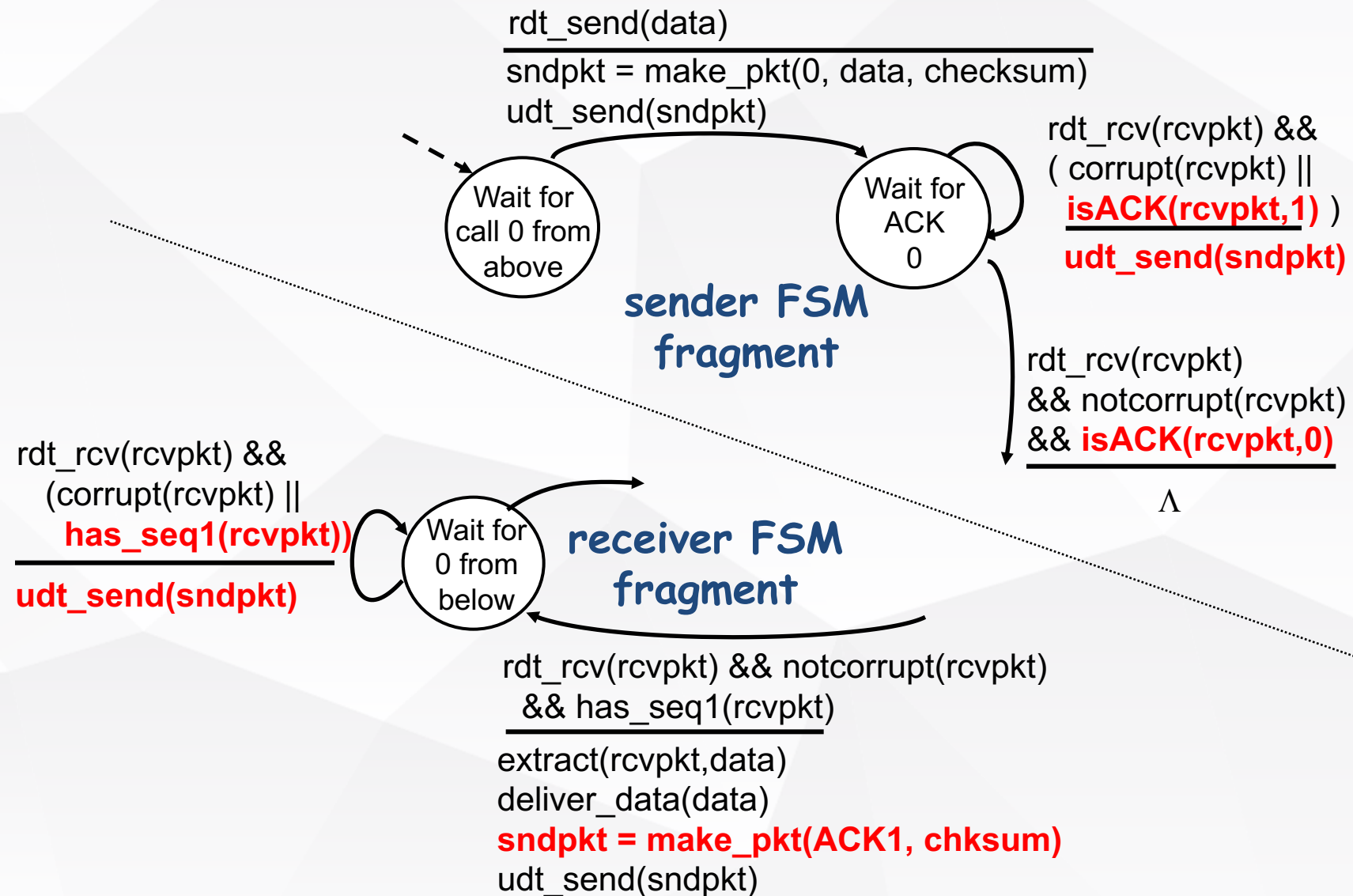
# RDT2.1: 管理出错的ACK/NAK, 接收方



# RDT2.2

- 和RDT2.1的功能相同，但仅使用ACK
  - 接收方正确接收一个包后，发送**ACK**
  - 在**ACK**包中，接收方必须通过序号指明是对哪个数据包的确认
- 重复的ACK包对发送方来说，和收到NACK的效果一样
  - 重发当前数据包

# RDT2.2: 无 NAK的协议

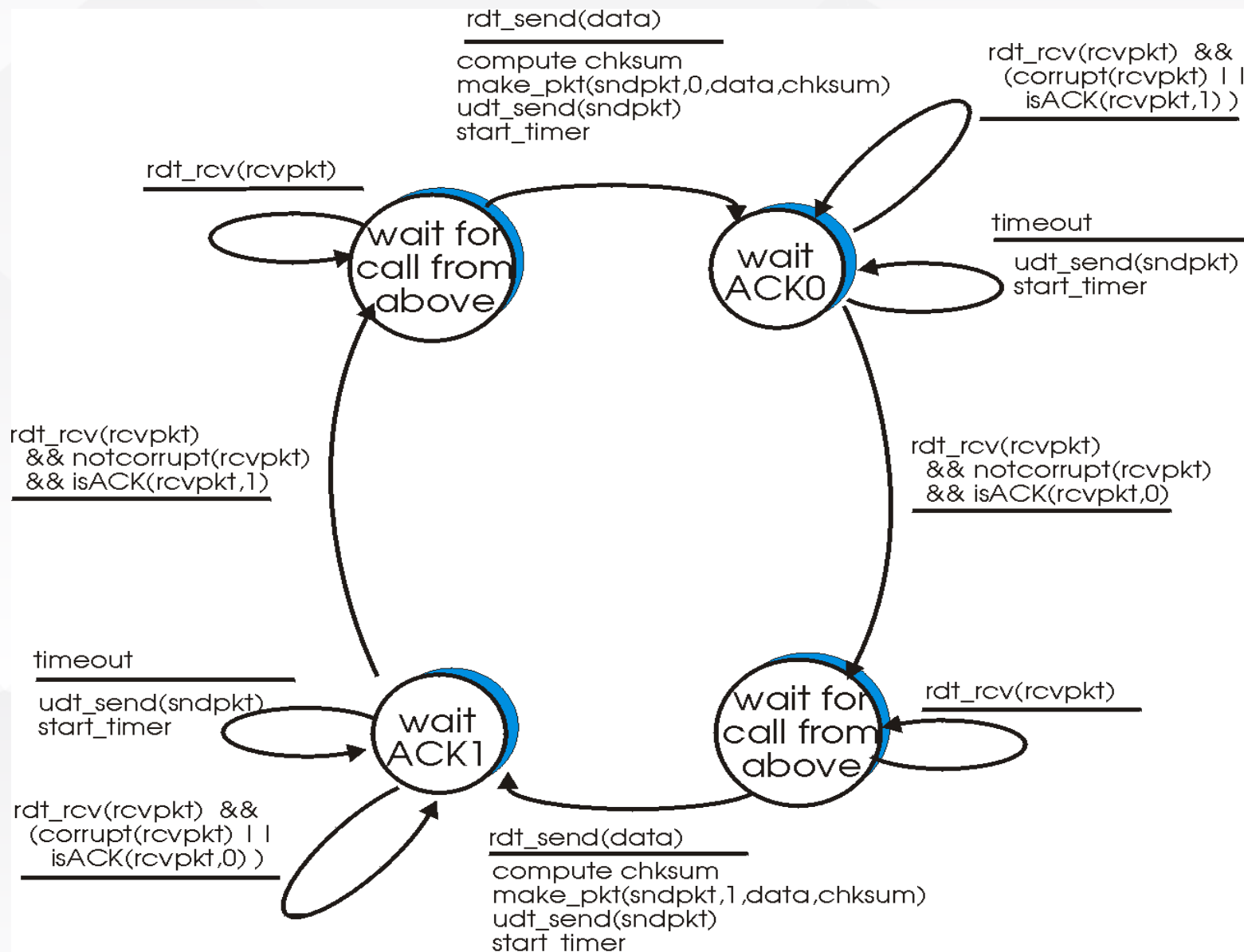




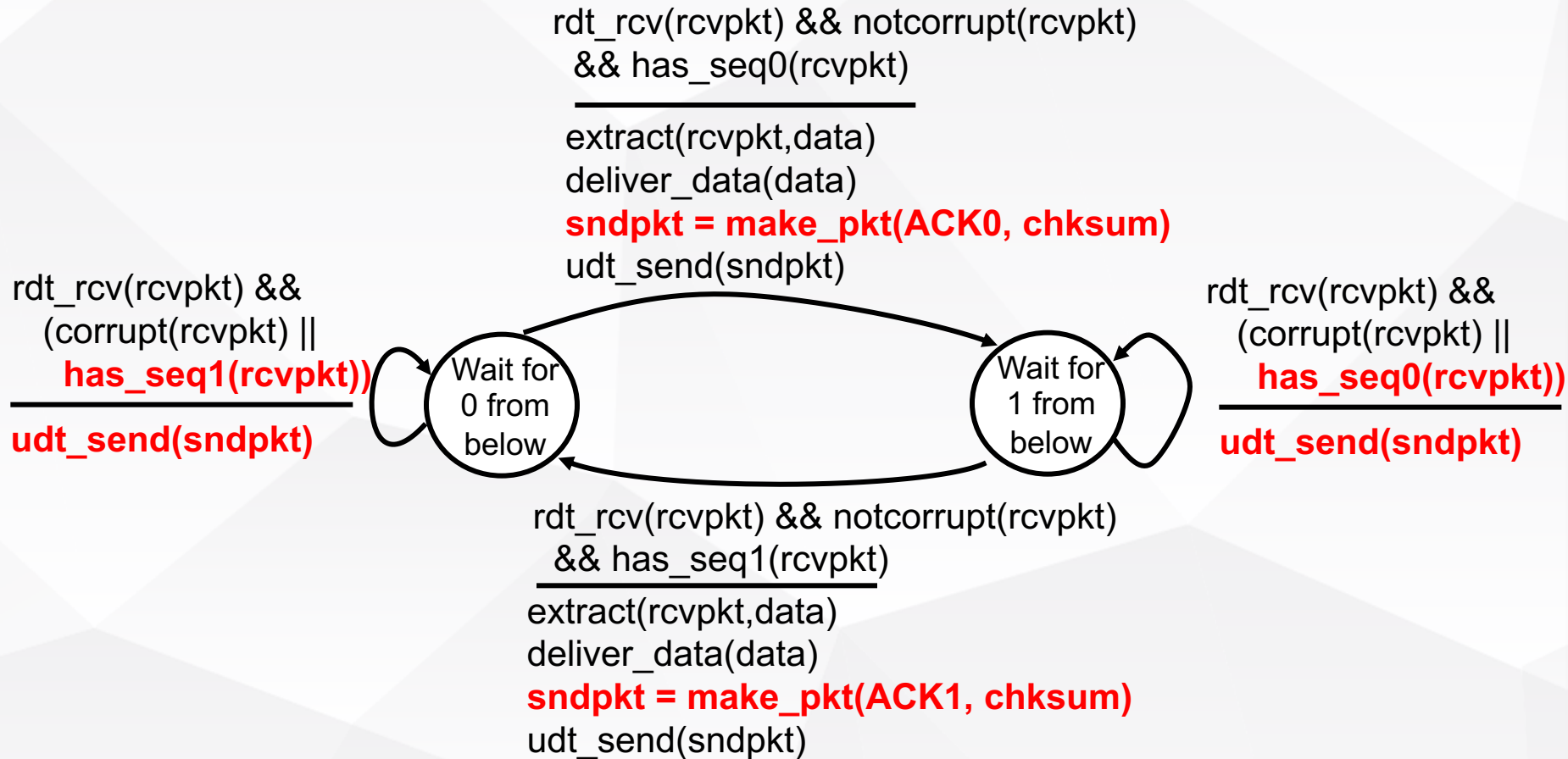
# RDT3.0: 通道上可能出错和丢失数据

- 新的假设: 底层信道可能丢包(数据或ACK)
  - checksum, seq. #, ACK, 重发机制会有帮助
- 如何处理数据丢失?
  - 发送方可以等待, 当某些数据或ACK丢失时, 进行重传
  - 想一想: 缺点?
- 方法: 发送方等待ACK一段“适当的”时间
- 如果在这段时间里没有收到ACK, 则进行重传
- 如果分组(或ACK)仅仅被延迟了(没有丢失):
  - 重传将导致重复, 但使用seq. #可以控制
  - 接收方必须指明被ACK分组所确认的seq #
- 需要进行倒计时

# RDT3.0 发送方



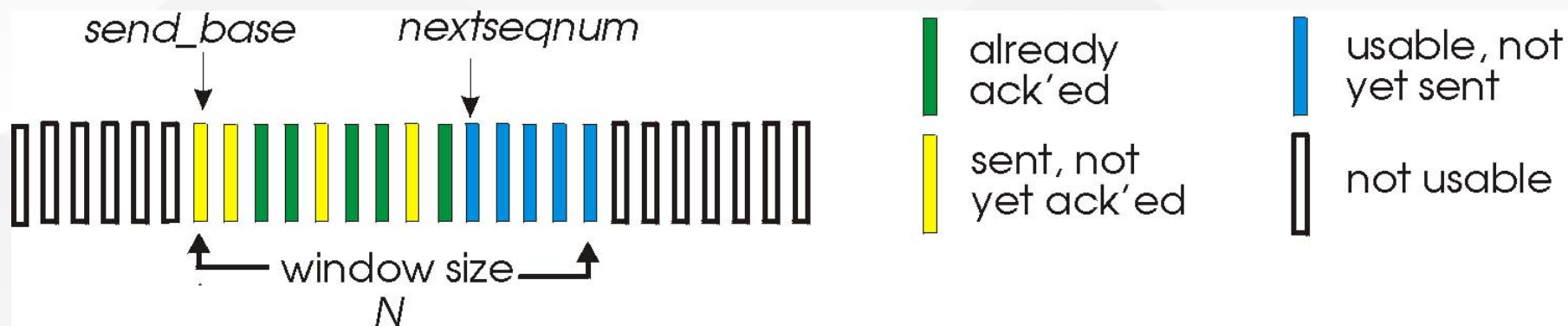
# RDT3.0: 接收方



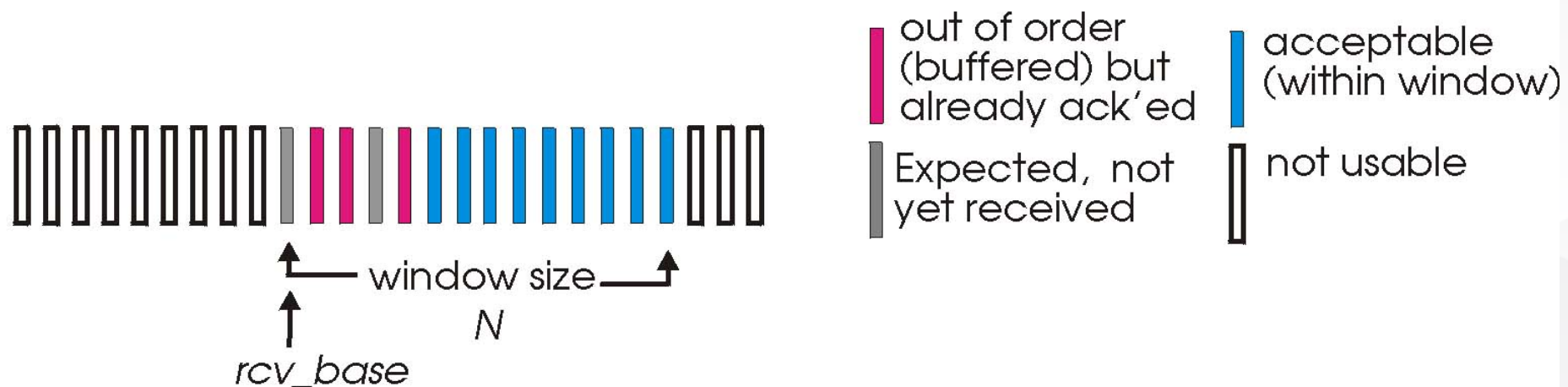
# 选择响应协议(Select Response)

- 接收方逐个对所有正确收到的分组进行应答
  - 对接收到的（失序）分组进行缓存，以便最后对上层进行有序递交
- 发送方仅对未收到应答的分组进行重发
  - 发送方为每个unACKed 分组设置计时器
- 发送方的窗口
  - N 个连续的 seq #'s
  - 同样对已发送的seq #'s但未收到ACK的分组进行限制

# 选择响应：发送方，接收方的窗口

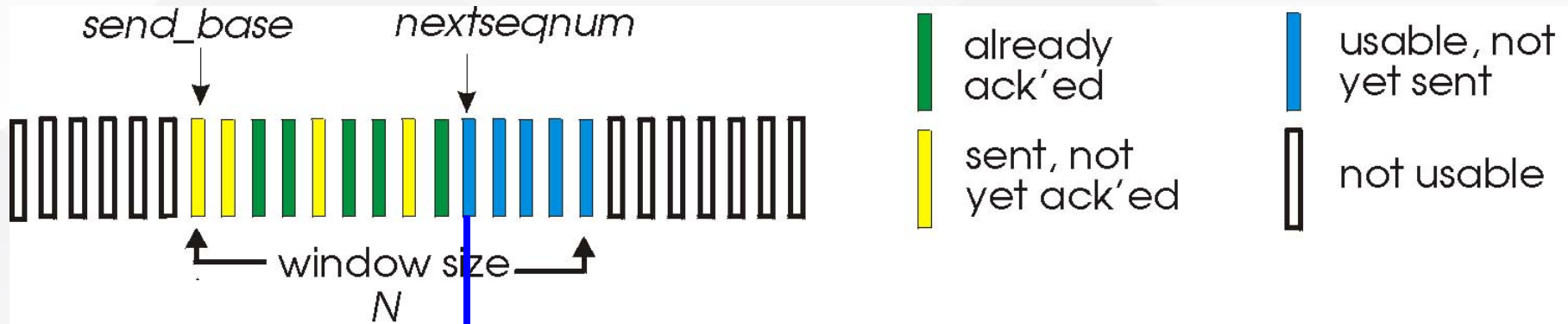


(a) sender view of sequence numbers



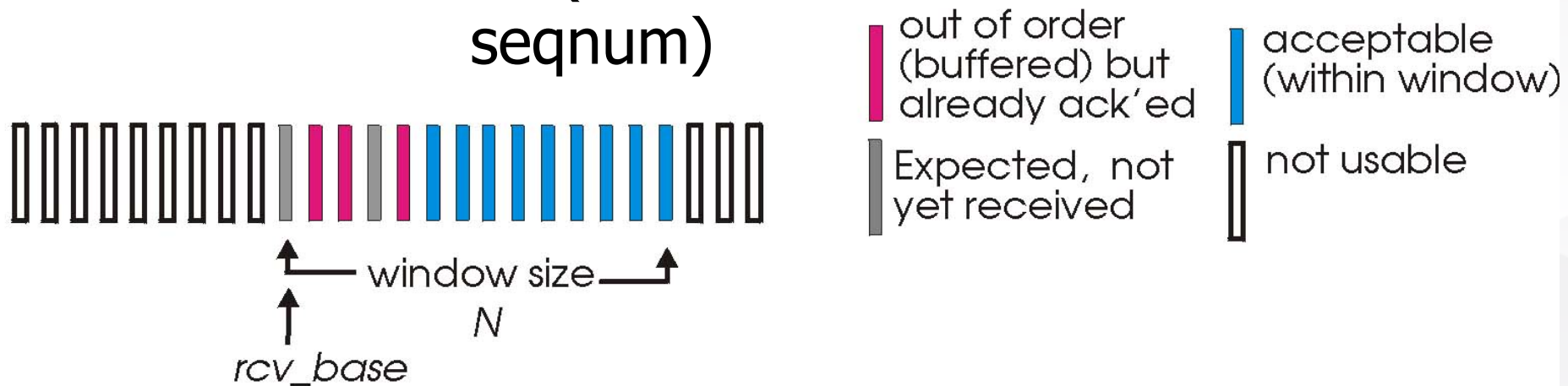
(b) receiver view of sequence numbers

# 发送方发送Pkt(nextseqnum)



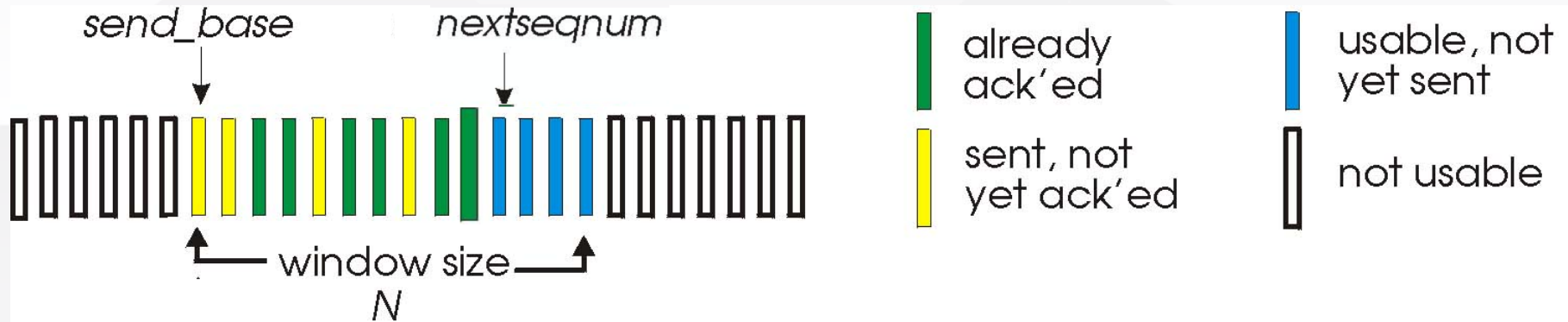
(a) sender view of sequence numbers

Pkt(next seqnum)

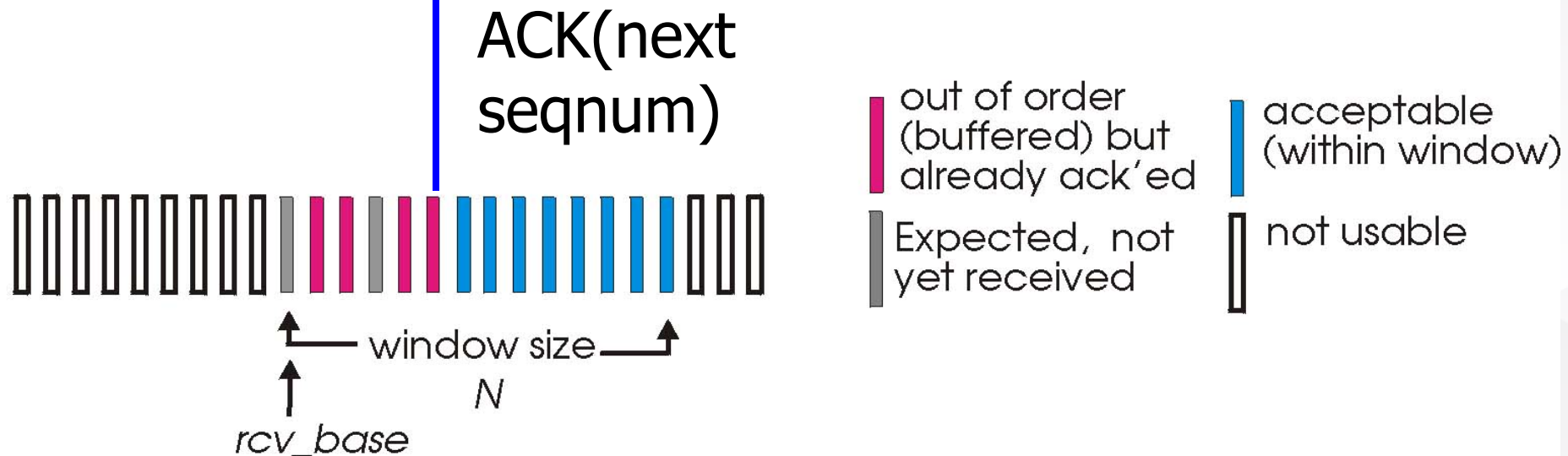


(b) receiver view of sequence numbers

# 接收方收到Pkt(nextseqnum), 应答

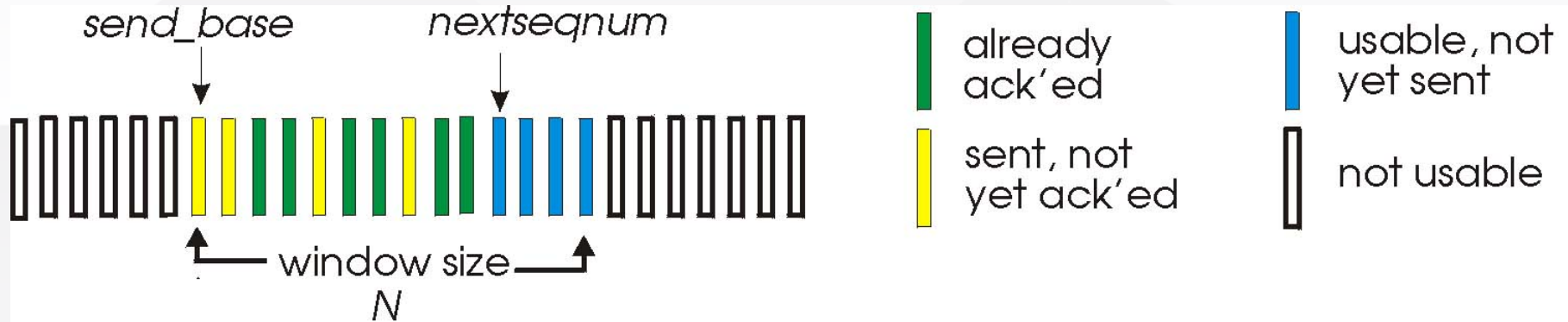


(a) sender view of sequence numbers

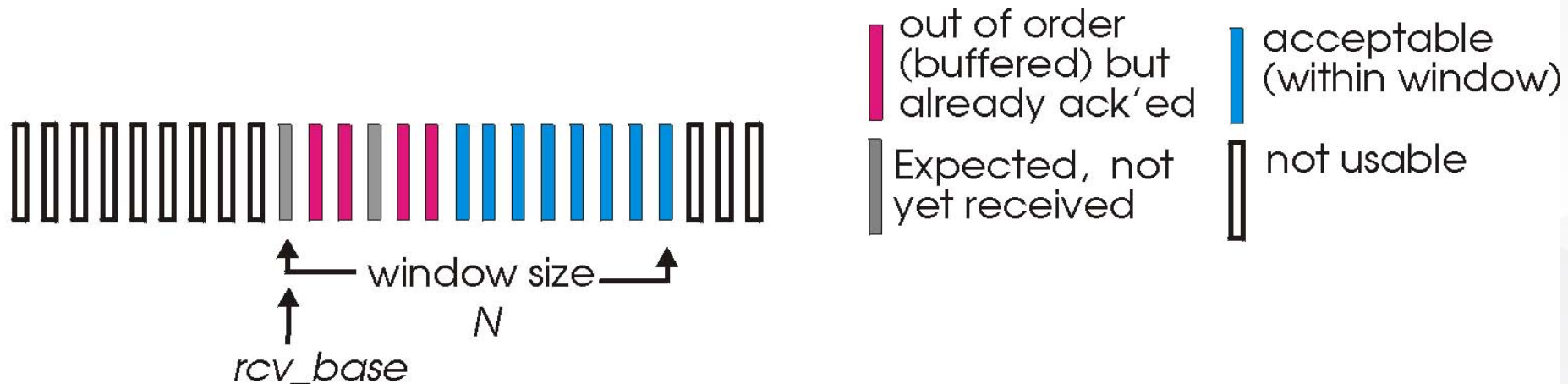


(b) receiver view of sequence numbers

# 发送方收到ACK(nextseqnum), 移动指针



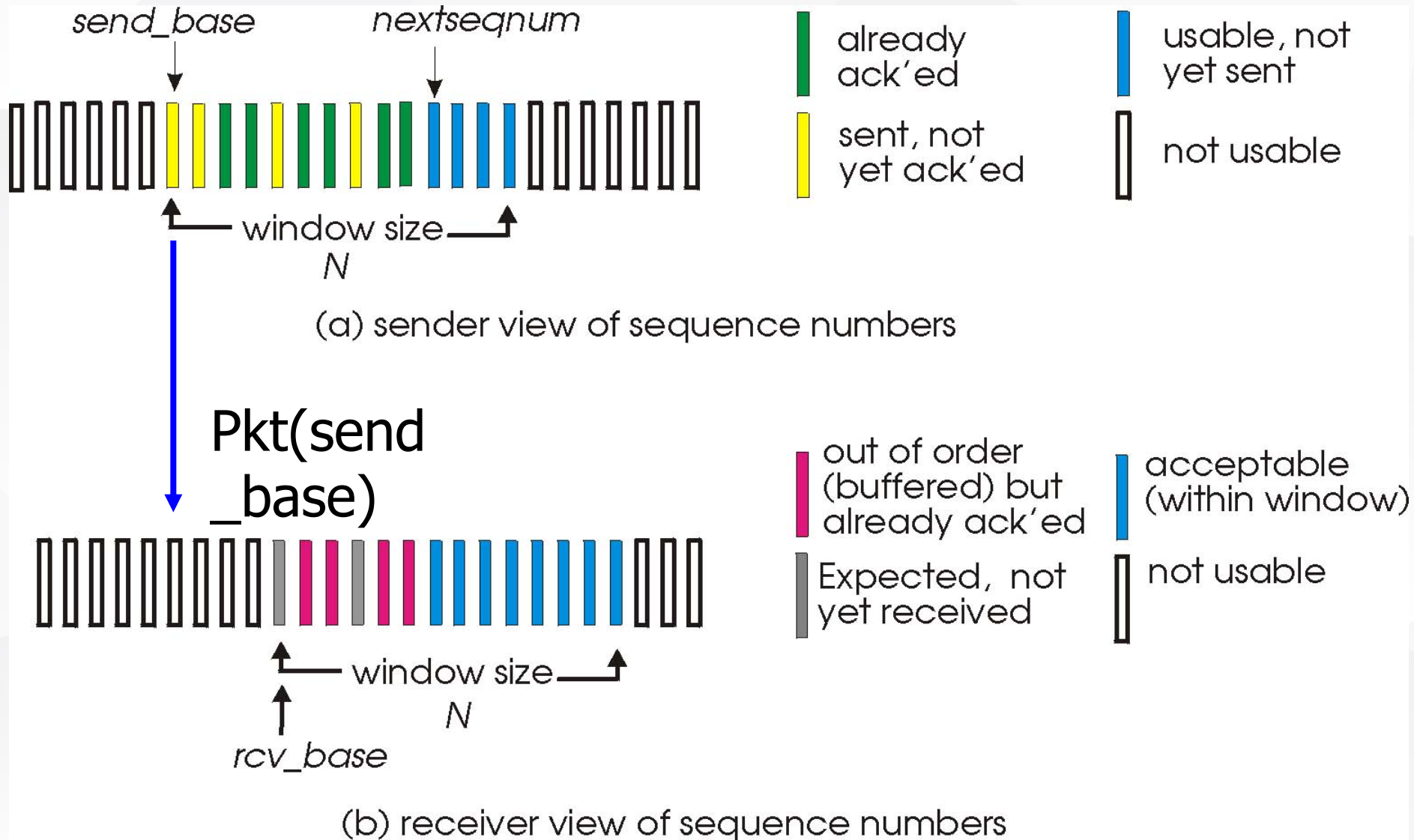
(a) sender view of sequence numbers



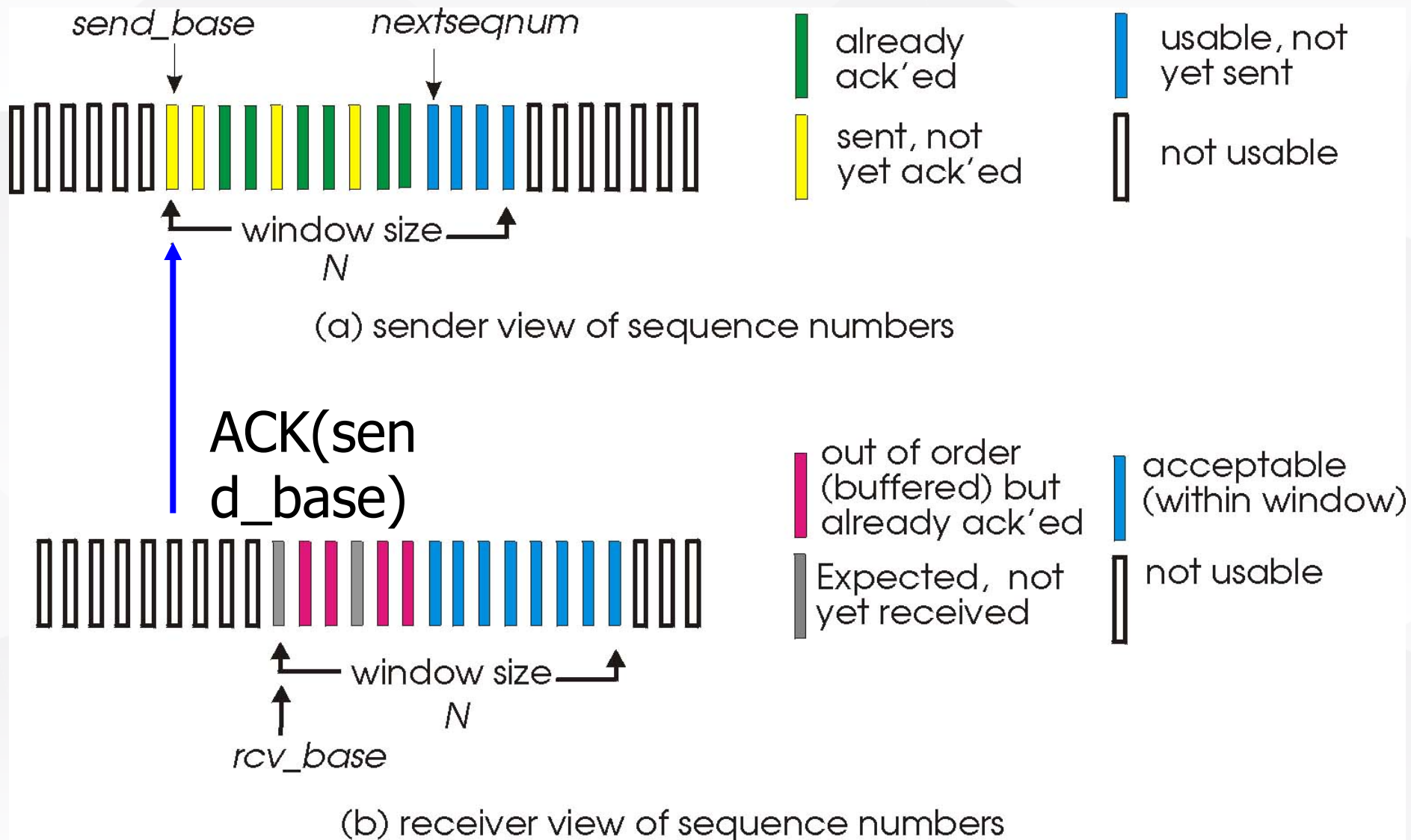
(b) receiver view of sequence numbers



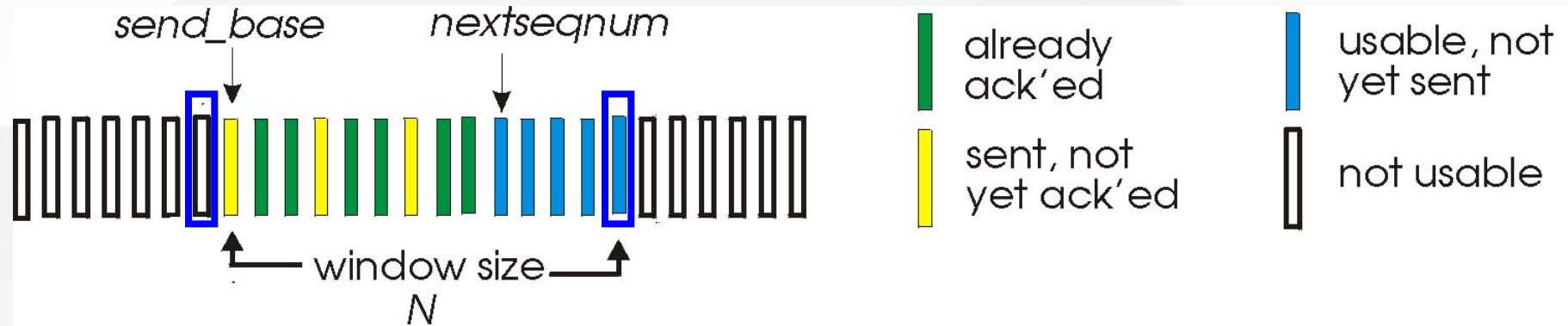
# 发送方重发Pkt (send\_base)



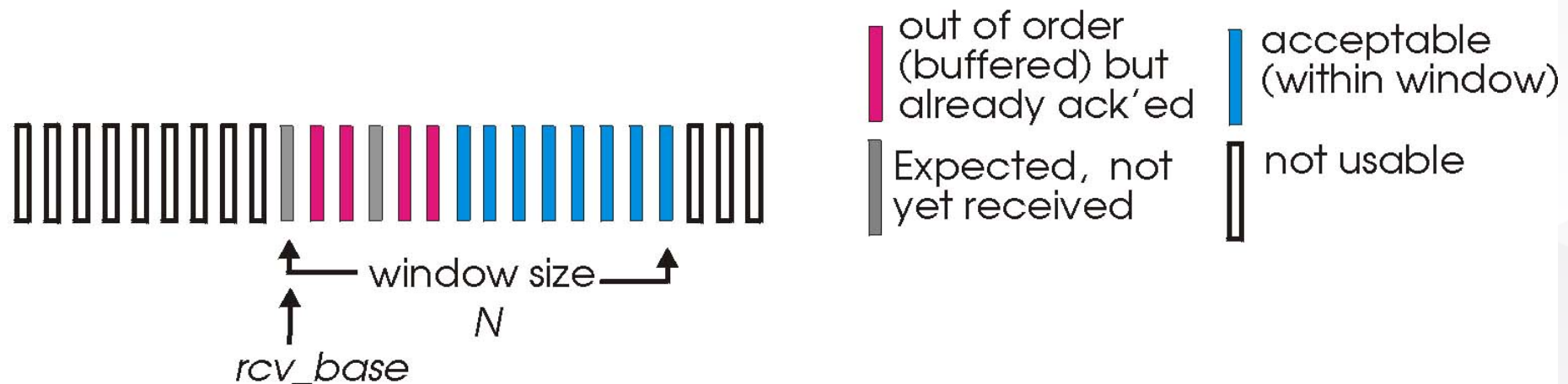
## 接收方收到Pkt (send\_base), 应答



# 发送方收到ACK (send\_base), 移动窗口

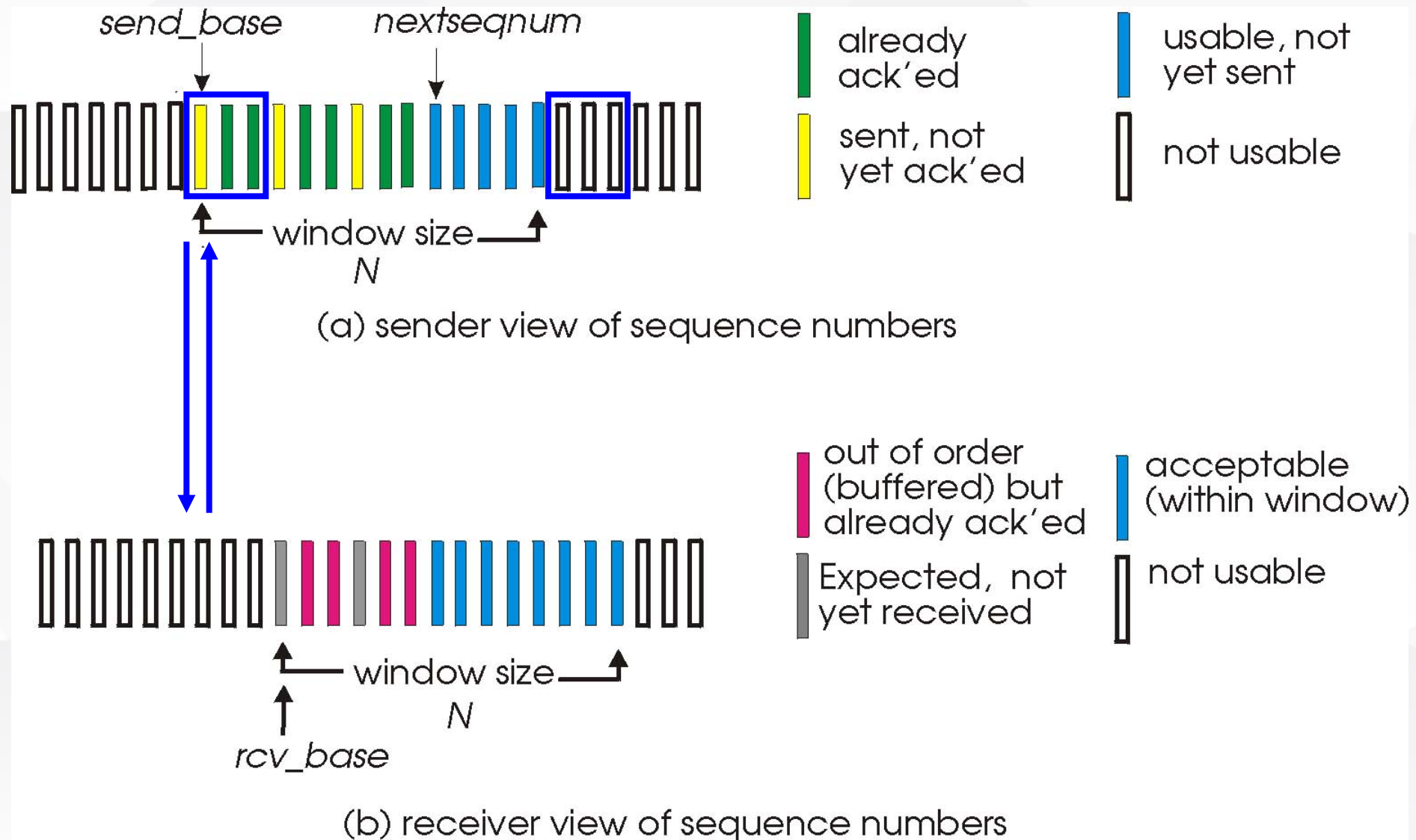


(a) sender view of sequence numbers

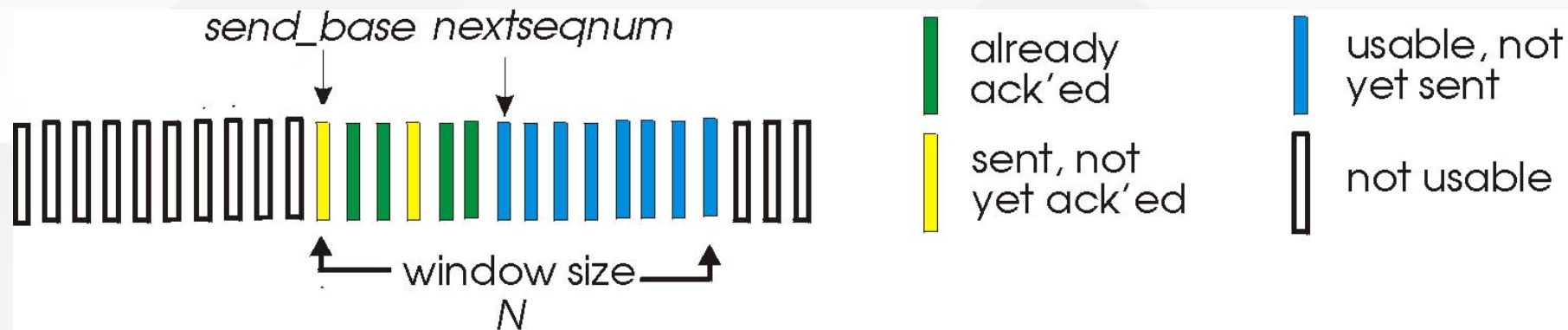


(b) receiver view of sequence numbers

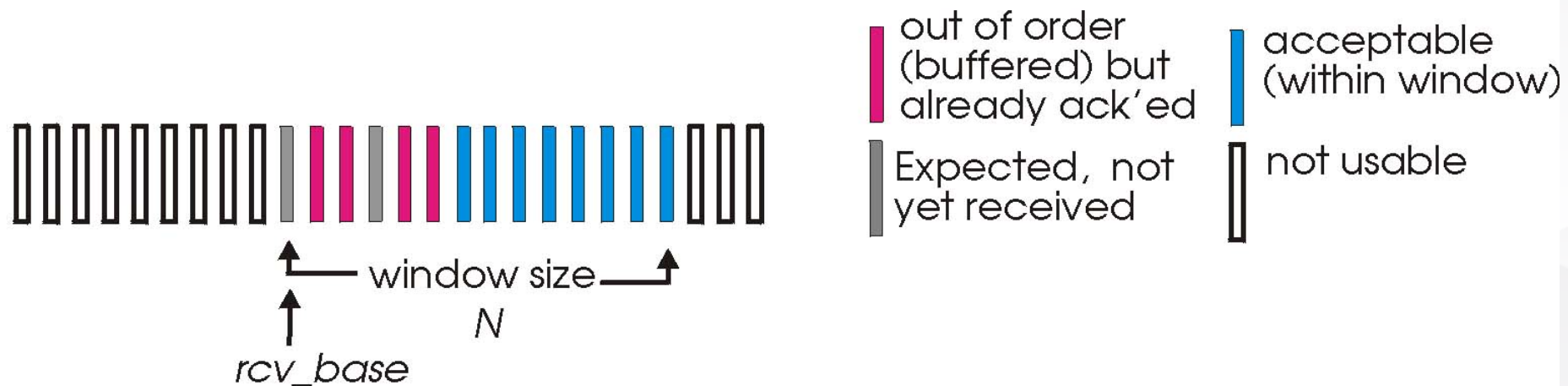
## 发送方发送新Pkt (send\_base), 接收方应答



## 发送方收到新的ACK (send\_base), 移动窗口

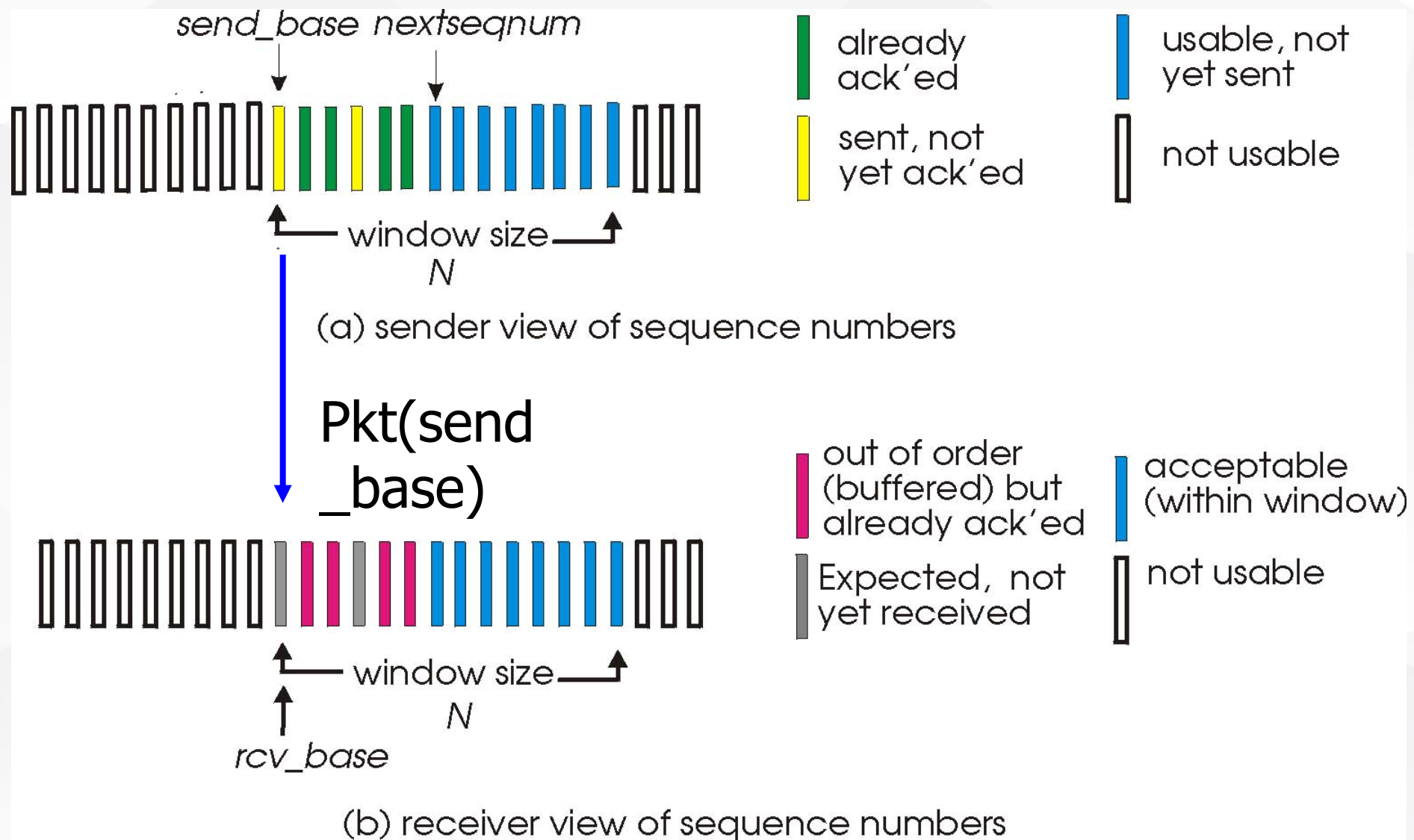


(a) sender view of sequence numbers



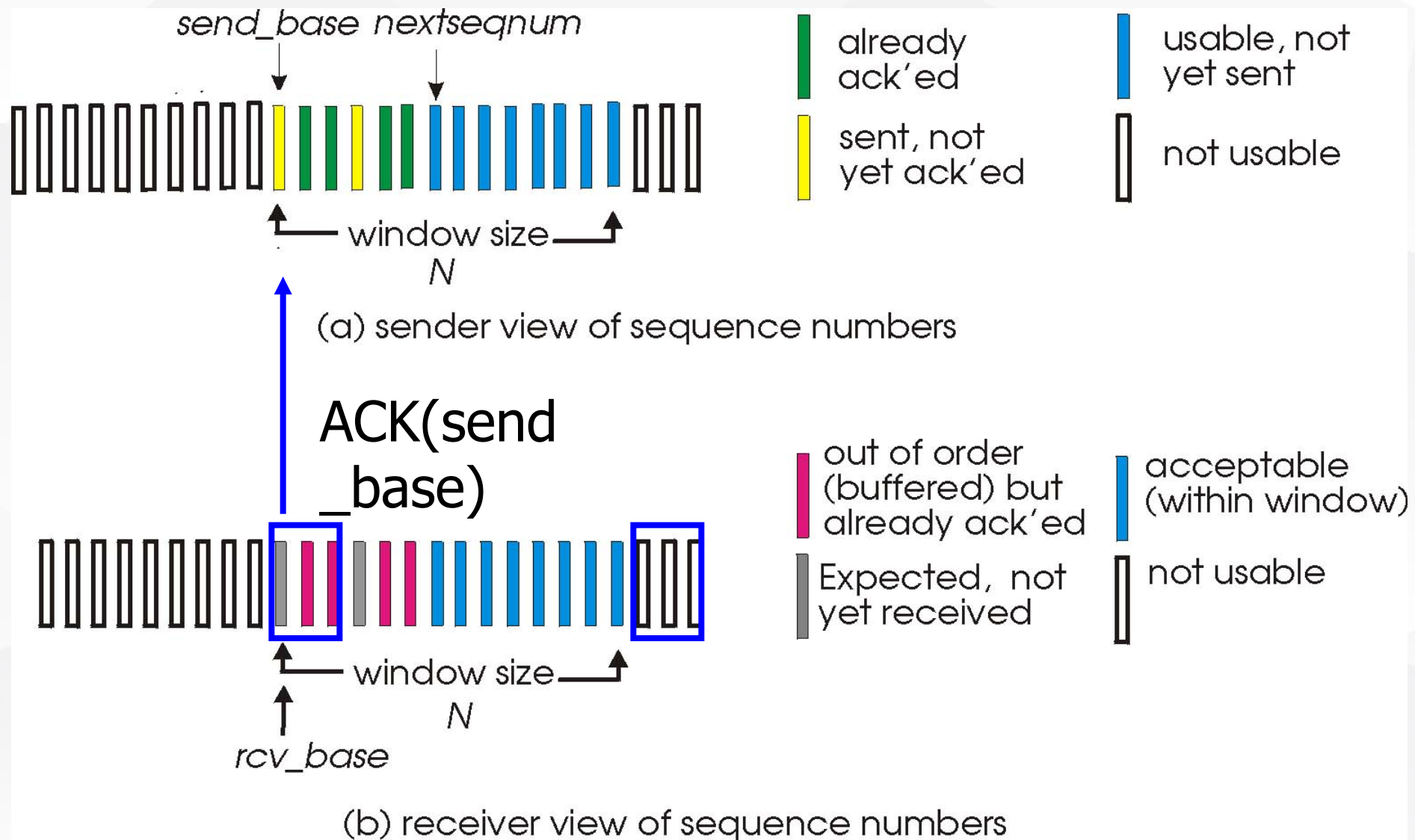
(b) receiver view of sequence numbers

# 发送方重发新的Pkt(send\_base)

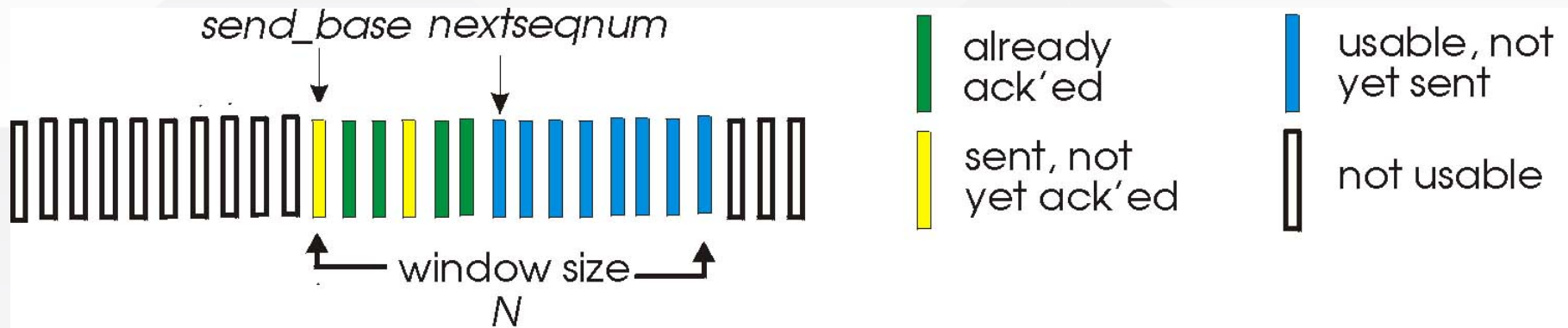




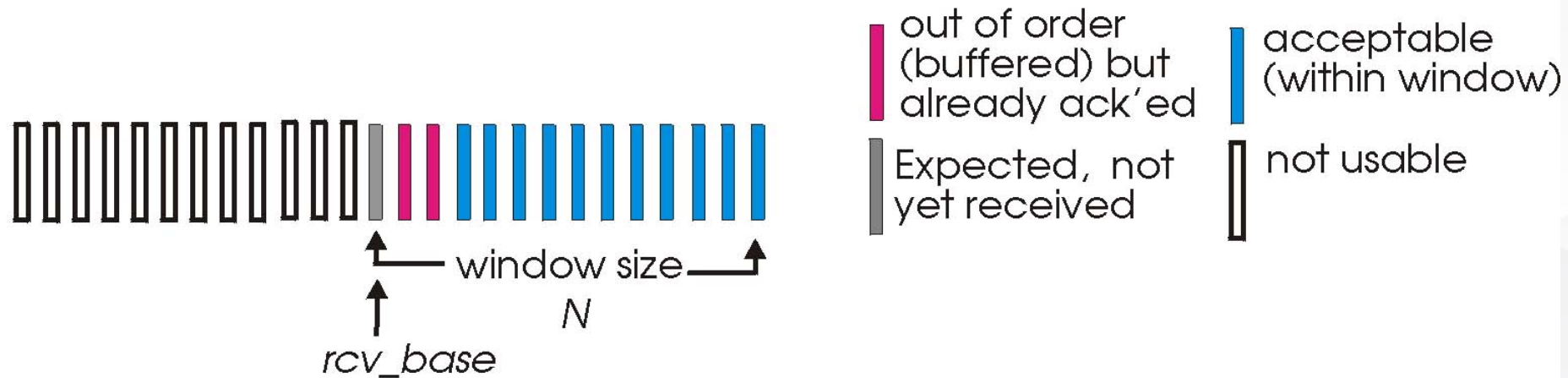
# 接收方收到,应答, 移动窗口



# 接收方移动窗口后



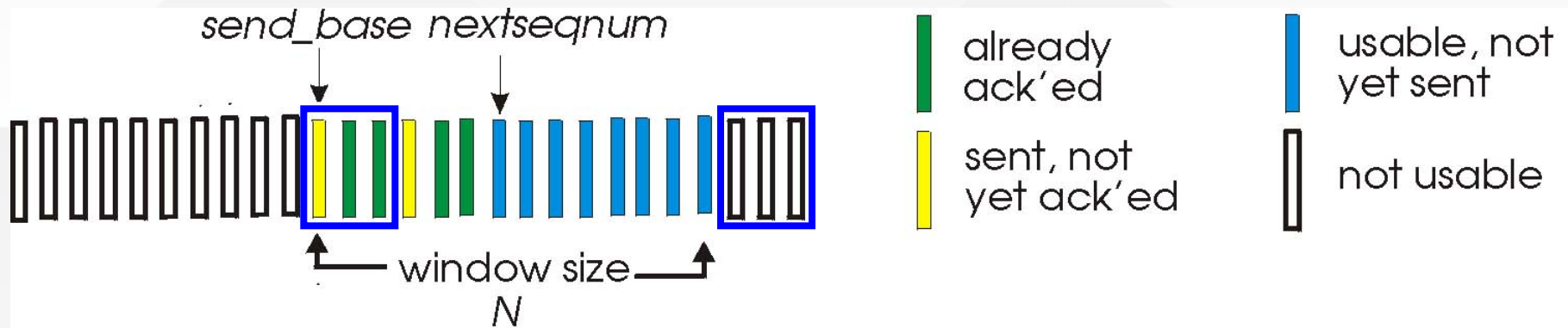
(a) sender view of sequence numbers



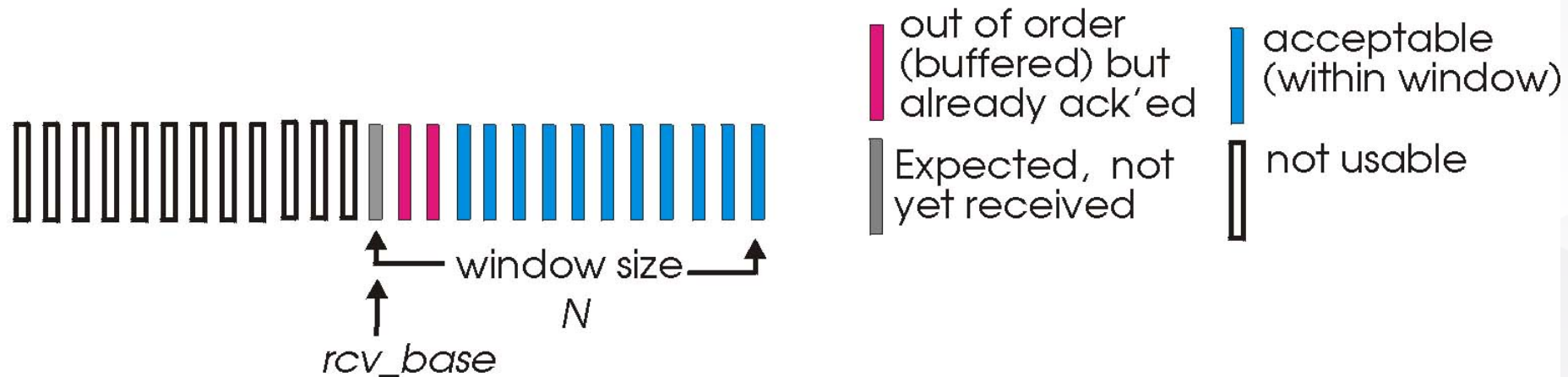
(b) receiver view of sequence numbers



# 发送方收到ACK, 移动窗口



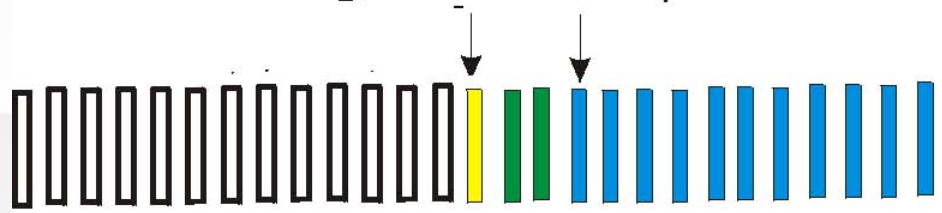
(a) sender view of sequence numbers



(b) receiver view of sequence numbers

# 发送方移动窗口后(一致状态)

*send\_base* *nextseqnum*



already  
ack'ed

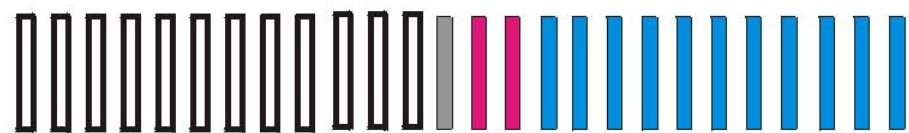
sent, not  
yet ack'ed

usable, not  
yet sent

not usable

— window size —  
*N*

(a) sender view of sequence numbers



out of order  
(buffered) but  
already ack'ed

Expected, not  
yet received

acceptable  
(within window)

not usable

— window size —  
*N*

*rcv\_base*

(b) receiver view of sequence numbers

# 选择响应协议

## 发送方

### 上层数据到达：

- 如果窗口中的下一个序号可用，发送分组

### timeout(n):

- 第n个计时器**超时**
- 重发分组n, 计时器复位

### ACK(n) 到达

[sendbase, sendbase+N]:

- 标记分组 n 已经收到
- 如n为unACKed分组中的最小值, 将窗口下沿前推到下一个unACKed seq #

## 接收方

### 分组n到达

[rcvbase, rcvbase+N-1]

- 发送 ACK(n)
- 失序: 缓存
- 有序: 递交到上层 (同时递交缓存中的其他有序分组), 将窗口左沿前推到下一个尚未收到的分组

[rcvbase-N, rcvbase-1]

- 虽然接收方曾经确认, 但仍然需要ACK(n)

### 其他情况:

- 忽略分组

---

# Thank You

